

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Industrial Problem Solving

COURSE CODE: CSA0904

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO4: Applet Programming-Applet Architecture - Simple Applet Display Methods
Event handling in Java - Delegation Event Model - Event Classes - Sources of Events - Event
Listener Interfaces -AWT Component and GUI Design : AWT Classes, Controls, Layout
Managers, and Menus.(BL3,BL4)
Assessment Tool Weightage: 20%

QUESTIONS: 2

Total Marks: 100

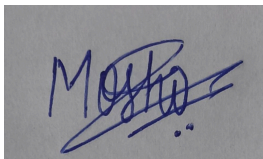
ANNEXURE A

Industrial Problem Solving

Code of Conduct:

*I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Moshika M', is written over a grey rectangular background.

RUBRICS FOR CALCULATION:

Criteria	Wt. %	Level 4 Exemplary 100% of weightage	Level 3 Proficient 75% of weightage	Level 2 Developing 50% of weightage	Level 1 Beginning 25% of weightage
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases
Total	100 %				

1. **Hospital Appointment Management System**
Design a Swing-based GUI application for a hospital to manage patient appointments, including booking, updating, and canceling schedules. Use AWT/Swing controls such as buttons, text fields, tables, and menus to facilitate user interaction. Implement event listeners to handle actions like appointment confirmation and deletion. Ensure the system can handle dynamic patient data efficiently. Evaluate different layout managers (FlowLayout, GridLayout, BorderLayout) for organizing complex UI components. Analyze how the design supports scalability when the number of patients increases. Propose improvements for better usability and real-time updates.
2. **Online Banking Interface**
Develop a GUI-based banking application that allows users to perform operations such as fund transfer, balance inquiry, and viewing transaction history. Use Swing components and event listeners to manage user interactions securely. Implement input validation for sensitive data like account number and amount. Ensure proper error handling for invalid transactions. Apply suitable layout managers to create a responsive and user-friendly interface. Analyze system responsiveness during multiple operations and suggest improvements. Demonstrate how secure input handling enhances reliability.
3. **Inventory Management Dashboard**
Create a GUI-based inventory management system using Swing components such as tables, forms, and menus to manage product details. Implement event handling for operations like adding, updating, and deleting products. Ensure efficient data display and manipulation for large inventories. Evaluate system scalability when handling thousands of records. Analyze usability aspects such as navigation, search functionality, and responsiveness. Suggest improvements like pagination or filtering for better performance. Justify your design choices based on real-world business requirements.
4. **Customer Feedback System**
Design an Applet-based GUI to collect customer feedback using input fields, rating options, and submission buttons. Implement event handling to process user inputs and dynamically update the display of feedback results. Ensure proper validation of inputs before submission. Analyze the limitations of Applet technology in modern applications, including security and browser compatibility issues. Evaluate the effectiveness of the chosen UI design and layout managers. Suggest modern alternatives such as web-based or desktop frameworks for improved performance and scalability.
5. **Ticket Booking System**
Develop a multi-window GUI application for ticket booking with features like seat selection, booking confirmation, and cancellation. Use Swing components such as buttons, panels, and menus along with appropriate layout managers to design the interface. Implement event handling to manage user interactions and ensure accurate seat allocation. Validate user inputs and handle exceptions during booking. Ensure smooth navigation between windows. Analyze user experience and identify areas for improving real-time interaction and responsiveness. Demonstrate the system with sample booking scenarios.

Q1 Ticket Booking System

50 marks

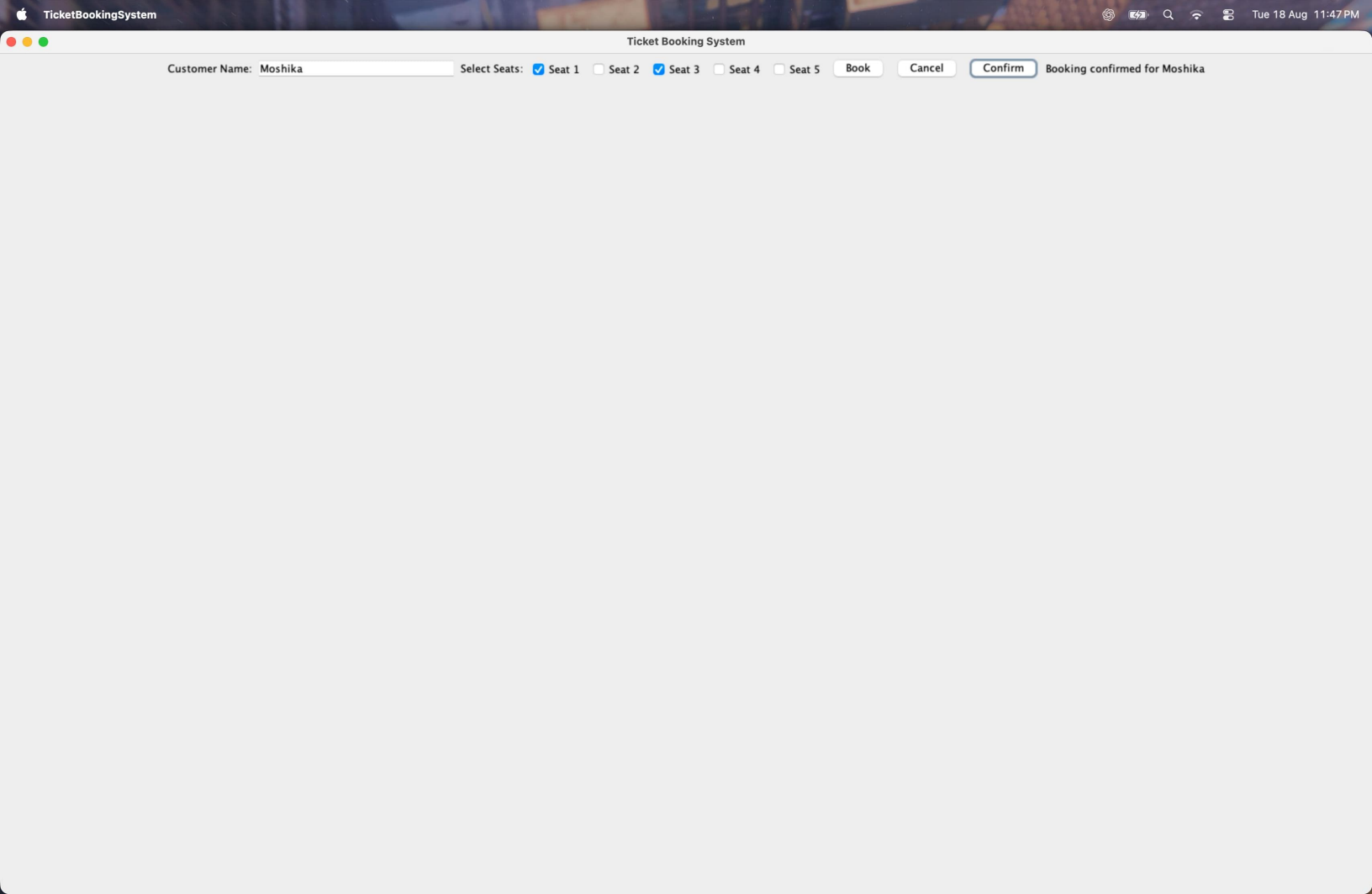
Develop a multi-window GUI application for ticket booking with features like seat selection, booking confirmation, and cancellation. Use Swing components such as buttons, panels, and menus along with appropriate layout managers to design the interface. Implement event handling to manage user interactions and ensure accurate seat allocation. Validate user inputs and handle exceptions during booking. Ensure smooth navigation between windows. Analyze user experience and identify areas for improving real-time interaction and responsiveness. Demonstrate the system with sample booking scenarios.

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 public class TicketBookingSystem extends JFrame implements ActionListener {
6
7     JButton book, cancel, confirm;
8     JCheckBox s1, s2, s3, s4, s5;
9     JTextField name;
10    JLabel result;
11
12    public TicketBookingSystem() {
13        setTitle("Ticket Booking System");
14        setSize(500, 400);
15        setLayout(new FlowLayout());
16
17        add(new JLabel("Customer Name:"));
18        name = new JTextField(20);
19        add(name);
20
21        add(new JLabel("Select Seats:"));
22
23        s1 = new JCheckBox("Seat 1");
24        s2 = new JCheckBox("Seat 2");
25        s3 = new JCheckBox("Seat 3");
26        s4 = new JCheckBox("Seat 4");
27        s5 = new JCheckBox("Seat 5");
28
29        add(s1);
30        add(s2);
31        add(s3);
32        add(s4);
33        add(s5);
34
35        book = new JButton("Book");
36        cancel = new JButton("Cancel");
37        confirm = new JButton("Confirm");
38
39        add(book);
40        add(cancel);
41        add(confirm);
42
43        result = new JLabel("");
44        add(result);
45
46        book.addActionListener(this);
47        cancel.addActionListener(this);
48        confirm.addActionListener(this);
49
50        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51        setVisible(true);
52    }

```





Customer Name: Moshika

Select Seats: ☒ Seat 1☐ Seat 2☒ Seat 3☐ Seat 4☐ Seat 5

Book

Cancel

Confirm

Booking confirmed for Moshika

Q2

Customer Feedback System

50 marks

Design an Applet-based GUI to collect customer feedback using input fields, rating options, and submission buttons. Implement event handling to process user inputs and dynamically update the display of feedback results. Ensure proper validation of inputs before submission. Analyze the limitations of Applet technology in modern applications, including security and browser compatibility issues. Evaluate the effectiveness of the chosen UI design and layout managers. Suggest modern alternatives such as web-based or desktop frameworks for improved performance and scalability.

```

1 import java.applet.Applet;
2 import java.awt.*;
3 import java.awt.event.*;
4
5 public class CustomerFeedback extends Applet implements ActionListener {
6
7     Run main | Debug main
8     public static void main(String[] args) {
9         Frame f = new Frame("Customer Feedback System");
10         CustomerFeedback applet = new CustomerFeedback();
11
12         applet.init();
13
14         f.add(applet);
15         f.setSize(500, 400);
16         f.setVisible(true);
17
18         f.addWindowListener(new WindowAdapter() {
19             public void windowClosing(WindowEvent e) {
20                 System.exit(0);
21             }
22         });
23
24         TextField name;
25         TextArea feedback;
26         Choice rating;
27         Button submit;
28         Label result;
29
30         public void init() {
31             setLayout(new FlowLayout());
32
33             Label title = new Label("Customer Feedback System");
34             name = new TextField(25);
35             feedback = new TextArea(5, 30);
36             rating = new Choice();
37             submit = new Button("Submit");
38             result = new Label("");
39
40             rating.add("Select Rating");
41             rating.add("1 - Very Poor");
42             rating.add("2 - Poor");
43             rating.add("3 - Average");
44             rating.add("4 - Good");
45             rating.add("5 - Excellent");
46
47             add(title);
48             add(new Label("Customer Name:"));
49             add(name);
50             add(new Label("Feedback:"));
51             add(feedback);
52             add(new Label("Rating:"));

```


Applet Viewer: CustomerFeedback.class

Customer Feedback System Customer Name:

Feedback:

The food is okayish

Rating: ..